Эпистемология и среда

ИИ-ассистенты для исследователя. AI4Science — неделя 1 из 12

Как устроен ИИ-ассистент исследователя и как его проверить?

А. П. Халов, О. М. Атаева

ФИЦ ИУ РАН МФТИ ШАД Яндекса

Осень 2026



План недели

О курсе

ИИ в науке: 2024–2026

Агент и рабочая среда

Как работает языковая модель

От модели к агенту

Написанный и порождённый текст

Петля недоверия

Практика и итоги

Пара 1

Курс, ИИ в науке сегодня, агент и рабочая среда

Цель курса

Цель — собрать и держать под контролем личную исследовательскую среду с ИИ-ассистентом на своём ноутбуке и пройти в ней академический цикл работы над своей задачей: от постановки до статьи.



- **Инференс** — сменный компонент: облачная модель, локальная модель.
- **Контекст** — данные; переносятся между моделями и остаются после курса.
- **Верификация** — внешняя по отношению к модели проверка результата.

Формат

- 12 недель, по две пары в неделю.
- Практика — самостоятельно между занятиями, всегда на собственном материале: своя задача, свои данные, свои статьи.
- После каждой лекции — **артефакт недели**: файлы, которые остаются в вашем проекте.
- Нагрузка вне занятий — 4–5 ч в неделю, на неделях 7–10 — до 6 ч.

Два равнозначных направления проверки:

- *вычислительное* — практика: эмпирическая верификация синтетических утверждений: сопоставление с наблюдением через код и тесты, численный эксперимент, проверку данных; основное для естественных наук (неделя 6);
- *формальное* — теория: верификация аналитических утверждений выводом: Lean 4, для задач с формализуемым ядром (неделя 7).

Философия курса: позитивизм

Позитивизм (О. Конт [48]): научное знание опирается на наблюдение.
Логический позитивизм (Венский кружок): осмысленное утверждение либо аналитическое, либо синтетическое и проверяемое опытом [44], [45].



Границы

- Общее утверждение не верифицируется конечным числом наблюдений, но может быть опровергнуто наблюдением — критерий фальсифицируемости К. Поппера [46]. Тест и эксперимент находят ошибки, но не доказывают их отсутствия.
- Вид утверждения определяет способ его проверки: аналитическое или синтетическое.

Композиция курса

модуль	недели				оценивается в
0. Введение	1 Эпистемология и среда				
I. Природа агента, контекст и память	2 Агент как функциональная система	3 Окно контекста и память	4 Онтологии и граф знаний	5 RAG и внешние графы знаний	**ДЗ 1** к неделе 6
II. Верификация	6 Вычислительная верификация		7 Декомпозиция проверки; Lean 4		**ДЗ 2** к неделе 9
III. Инструменты, среда и масштаб	8 Свой сервер MCP и удалённая среда		9 Многоагентные системы и инженерия циклов		**ДЗ 3** к неделе 12
IV. Полный цикл и представление	10 От постановки до статьи		11 Рецензирование		**Статья** конец недели 10
V. Своя среда и закрытие курса	12 Среда без облака и локальные модели; мини-конференция				

Подробный состав недель — в силлабусе; схемы и обоснования — в концептуальном атласе курса.

Оценивание и сроки

Работа	Недели	Срок	Баллы
ДЗ 1. Среда и контекст	1–5	к неделе 6	20
ДЗ 2. Проверка	6–7	к неделе 9	20
ДЗ 3. Инструменты и масштаб	8–9	к неделе 12	20
Статья 2–4 с. (с рецензиями и докладом)	1–12	конец недели 10	40

- Итог: ≥ 80 — «отлично», ≥ 60 — «хорошо», ≥ 40 — «удовлетворительно».
- Работа после срока — половина баллов; досдача домашних работ — две недели после курса.
- Срок статьи жёсткий: от него зависят слепое рецензирование и мини-конференция.

Ресурсы курса

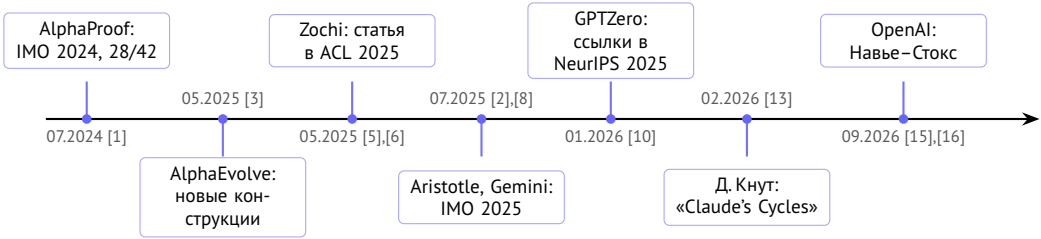
- **Учебный агент AI4Science** (открытый код):
  `https://github.com/andkhalov/AI4Science`
- **Доступ к моделям**: учебный грант Yandex AI Studio;
  ключ и идентификатор каталога предоставляются (лимит 3М).
- **Служебные сервисы курса**: проверка Lean, поиск по Mathlib;
  подключены к агенту через MCP.
- **Общая папка курса**: слайды, артефакты, шаблоны —
  `https://disk.360.yandex.ru/d/vv8ntiazDCWEWQ`
- **Чат курса** в Telegram: вопросы, объявления, изменения расписания.
- **Удалённый сервер курса** — с недели 8.

Демонстрация

ИИ-ассистент в исследовательской практике

- Два исследовательских проекта: журналы экспериментов в TensorBoard.
- Учебная платформа ai4math с контролируемой средой для студентов.
- Применение ИИ в задачах ОКТЧ.
- Формальная верификация — одно из направлений проверки (нед. 7)

Хронология: результаты с участием ИИ-систем



- Для каждого результата разбираем: кто порождал, чем проверяли, кто отвечает за вывод.
- Утверждения компаний о своих системах приводятся с указанием источника.

AlphaProof и Gemini Deep Think: IMO 2024 и 2025

AlphaProof, IMO 2024 [1]

- Агент, обучаемый с подкреплением на миллионах автоматически формализованных в Lean задач.
- Условия задач формализованы экспертами вручную. Формальные доказательства P1, P2, P6 — по 2–3 дня вычислений на задачу; P4 решена системой AlphaGeometry 2.
- Итог 28 из 42 баллов — уровень серебряной медали. Доказательства проверены ядром Lean и оценены Т. Гауэрсом и Дж. Майерсом.

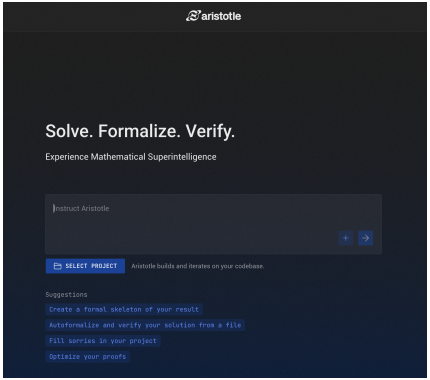
Gemini Deep Think, IMO 2025 [2]

- 5 из 6 задач (35 баллов) на естественном языке, в пределах 4,5 ч; решения оценены координаторами IMO.

Главное

Проверка устроена по-разному: ядро Lean для формальных доказательств [1], жюри олимпиады для текстовых решений [2].

Aristotle (Harmonic): формальные решения IMO 2025



Интерфейс сервиса:
`https://aristotle.harmonic.fun/`

- 5 из 6 задач IMO 2025; для четырёх негеометрических — доказательства на Lean 4 без `sorry`; геометрическая задача решена отдельным решателем вне Lean [8].
- Условия переведены в Lean вручную; по описанию авторов, это единственный шаг ручной формализации [8].
- Сервис доступен через веб-интерфейс и API.
- При формализации задач Эрдёша основным источником ошибок оказалось несоответствие формальной постановки исходной задаче [9].

Главное

Проверка Lean подтверждает доказательство относительно записанной постановки. Соответствие постановки исходной задаче проверяет человек.

AlphaEvolve: порождение программ и автоматический оценщик

- Эволюционный агент: языковые модели предлагают изменения программы, автоматический оценщик задаёт функцию качества [3].
- Результаты по сообщению авторов [3]: тензорное разложение ранга 48 для умножения комплексных матриц 4×4; конфигурация из 593 касающихся сфер в размерности 11 (прежняя оценка 592).
- Б. Георгиев, Х. Гомес-Серрано, Т. Tao, А. Вагнер: 67 задач анализа, комбинаторики, геометрии, теории чисел; лучшие известные конструкции воспроизведены в большинстве случаев, улучшены в нескольких [4].
- Для задачи Какеи над конечными полями: конструкцию нашёл AlphaEvolve, доказательство получил Deep Think, формализацию в Lean выполнил AlphaProof; это удавалось, когда нужные факты уже были в Mathlib [4].

Ограничение, отмеченное авторами [4]

«Cheating phenomenon»: система находит дефекты в постановке или оценщике и выдаёт их за решение. Надёжность результата определяется качеством проверяющей функции.

Zochi: статья, подготовленная ИИ-системой

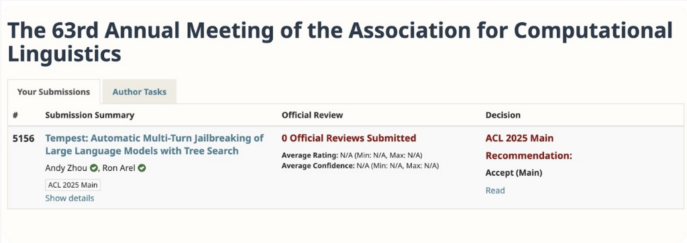


Figure 1. Zochi's paper, Tempest, accepted to the ACL 2025 main conference, as shown in the author submission portal.

- По сообщению Intology, статья Tempest [5] подготовлена системой Zochi и принята в основную программу ACL 2025 с мета-оценкой 4. Люди задали область исследования, проверили результаты и код, подготовили рисунки и ответ рецензентам [6].
- Правила ACL: генеративная модель не может быть соавтором [7].
  На 14.09.2026 статья в ACL Anthology не найдена.

Рисунок: Intology, «Zochi publishes A* paper» [6].

Сгенерированный текст в статьях и рецензиях

Ссылки в принятых статьях NeurIPS 2025 [10], [11]

- Принято 5290 статей из 21 575 подач [11].
- GPTZero проверила 4841 принятую статью: автоматический поиск ссылок, которых нет в открытых источниках, затем ручная проверка. Подтверждено 100 таких ссылок в 51–53 статьях [10].
- Отчёт выпущен коммерческим разработчиком инструмента; доля ложных срабатываний автоматического этапа не раскрыта.

Текст рецензий [12]

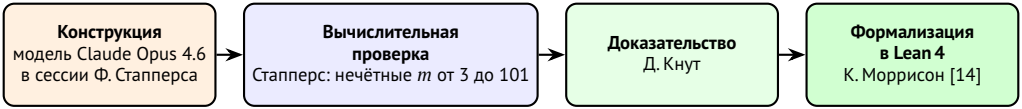
- Оценка на уровне корпуса методом максимального правдоподобия: доля текста рецензий, существенно изменённого языковыми моделями, — от 6,5 % (CoRL 2023) до 16,9 % (EMNLP 2023).
- Доля выше в рецензиях с низкой заявленной уверенностью и поданных незадолго до срока.

Главное

Каждая из этих статей прошла рецензирование. Сгенерированная ссылка выглядит так же, как настоящая, пока её не проверили по источнику.

Д. Кнут, «Claude's Cycles»: этапы и ответственные

Задача: разложить дуги орграфа на m^3 вершинах ijk (из каждой вершины — дуги в i^+jk, ij^+k, ijk^+, где $i^+ = (i+1) \bmod m$) на три ориентированных цикла длины m^3 [13].



- Конструкция для нечётных m получена за 31 итерацию, примерно за час; Стапперсу приходилось многократно напоминать модели записывать ход работы [13].
- Проверка программой на конечном диапазоне m не заменяет доказательства; доказательство написал Кнут [13].
- В заметке для каждого этапа указан исполнитель: модель или человек.

OpenAI: взрыв решений уравнений Навье–Стокса

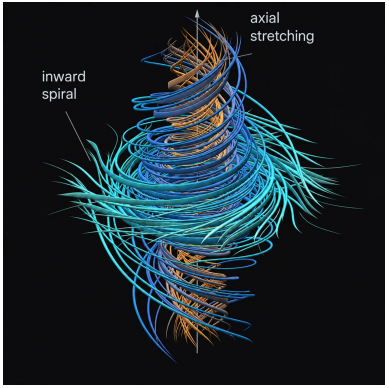


Схема течения: спираль к оси и осевое растяжение.

Препринт «Finite time blowup for Navier–Stokes», **166 страниц**, 08.09.2026 [16]. Теорема 1.1 (обозначения статьи): для любого $\nu > 0$ существуют сила $f \in C_c^\infty(\mathbb{R}^3 \times (0,\infty);\mathbb{R}^3)$ и гладкие поля u, p на $\mathbb{R}^3 \times [0,1)$:

$$\partial_t u + (u\cdot\nabla)u - \nu\Delta u + \nabla p = f, \quad \nabla\cdot u = 0, \quad u(\cdot,0) = 0,$$

$$\sup_{0\le t<1} \|u(t)\|_{L^2} < \infty, \qquad \limsup_{t\uparrow 1} \|u(t)\|_{L^\infty} = \infty.$$

Авторы относят результат к альтернативам (C) и (D) постановки Ч. Феффермана [18].

Строка авторов в препринте — «OpenAI». Опубликована Lean-формализация: `sorry_count : 0`; формальная постановка адаптирована из проекта Formal Conjectures [17].

Навье–Стокс: процесс по описанию компании и статус

Процесс, по сообщению OpenAI [15]

- Система координирующихся агентов на внутренней модели; доступ к кэшированной копии интернета и к выполнению кода.
- В группе, получившей решение, — порядка 10 000 одновременно работающих агентов; решение — примерно через 88 ч после запуска первых агентов; Lean-формализация — ещё 17 ч.
- Промежуточные выводы групп агентов сводились с помощью Codex и подавались агентам в следующих запросах.

Статус на 14.09.2026

- Параллельно Л. Альпёге и Т. Бакмастер опубликовали доказательства взрыва с гладкой внешней силой для уравнения Эйлера и других систем; OpenAI признала их приоритет в задаче Эйлера [15], [20].
- Clay Mathematics Institute: процедура оценки «deliberately unhurried» [19]; правила премии требуют публикации, двух лет и признания сообщества [23].
- Независимой экспертной проверки 166-страничного текста не опубликовано. 25 лауреатов Филдсовской премии выпустили заявление о расхождении целей ИИ-компаний и математического сообщества [22].

Что общего у этих результатов

AlphaProof [1]

порождено: формальные доказательства

проверено: ядро Lean; эксперты

Aristotle [8]

порождено: доказательства на Lean

проверено: ядро Lean; постановку записал человек

AlphaEvolve [3], [4]

порождено: программы-конструкции

проверено: автоматический оценщик; доказательство; Lean

Кнут [13]

порождено: конструкция

проверено: программа на конечном диапазоне; доказательство; Lean

Навье–Стокс [15]–[17]

порождено: доказательство, текст, Lean-код

проверено: Lean относительно формальной постановки; экспертиза не завершена

Zochi [5], [6]

порождено: статья

проверено: проверка людьми; рецензирование

есть исполнимая проверка: ядро Lean, программа, оценщик только проверка людьми

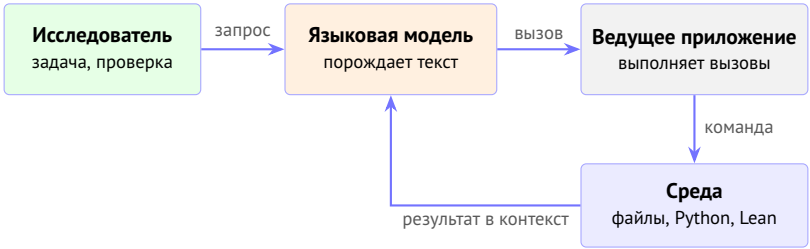
- Порождение становится дешёвым; узким местом становится проверка.
- Сильнее всего защищены результаты, прошедшие исполнимую проверку.
- Правила авторства и оценки таких результатов формулируются заново [7], [22], [23].

Всё можно рассматривать как код

статья	**данные**	**эксперимент**	**утверждение**	**договор с агентом**	**журнал работы**
`paper.tex` или .md	`data.csv` и схема	`run.py` params.yaml	`claim.lean`	`AGENT.md`	`diary.md`
проверка: сборка, сверка ссылок	проверка: схема, единицы, диапазоны	проверка: повторный запуск	проверка: ядро Lean	проверка: чтение, история изменений	проверка: история в git

- Текстовые файлы читает и изменяет агент, их версии хранит git, изменения видны построчно.
- Модели, обученные на коде, работают с такими файлами так же, как с программами.
- Тяжёлые форматы (`.docx`, `.pptx`) переводятся в лёгкие (`.md`, `.csv`, `.tex`) на входе и собираются обратно на выходе.

Агент на одном слайде



- Модель порождает текст; часть текста — вызовы инструментов в заданном формате.
- Ведущее приложение (в курсе — Goose [37]) выполняет вызов в среде и возвращает результат модели.
- Цикл повторяется, пока модель не выдаст ответ без вызова.

А теперь – демонстрация!

Учебный проект: AGENT.md, diary.md, data/measurements.csv – 12 измерений V (л), p (кПа) при $T = 293{,}15$ K. Проект под git.

```
$ cd ~/demo_boyle
$ ai4science --mode approve
> По данным data/measurements.csv проверь, выполняется ли закон
  Бойля-Мариотта: посчитай p·V для каждой строки, оцени среднее и
  относительный разброс, построй график p от 1/V в results/.
  Итог и способ проверки запиши в diary.md.
```

Режим approve: каждое действие агента выполняется после подтверждения, поэтому виден каждый шаг.

На что смотреть:

- какие команды выбирает агент и в каком порядке;
- какие файлы он создаёт и изменяет (git status, git diff);
- что записано в журнал как «способ проверки».

Установка агента

Windows 10/11

```
git clone https://github.com/andkhalov/AI4Science.git
cd AI4Science
setup.bat
```

`setup.bat` находит Python 3.10+, проверяет git, ставит зависимости и Goose, добавляет команду `ai4science` в PATH. При неудаче выводит запасной путь через WSL2.

macOS и Linux

```
git clone https://github.com/andkhalov/AI4Science.git
cd AI4Science && ./setup.sh
```

Удалённый сервер

```
ssh user@host
# далее как в macOS и Linux
```

Агент работает на сервере, файлы проекта — там же. Сессии, переживающие обрыв связи, — неделя 8.

Ключ и проверка

- Мастер установки записывает ключ и каталог Yandex AI Studio в файл `.env` в папке агента; `.env` не публикуется и не добавляется в git.
- `ai4science doctor` — проверка: модель, Goose, инструменты, сервис Lean.
- Удаление без следов в системе — раздел «Удаление» в README репозитория.

Сессия агента: модели и команды

Команды в сессии

`/model qwen235`	сменить модель
`/model`	текущая модель
`/status`	модель, режим, токены, контекст
`/mode approve`	режим подтверждения
`/plan <задача>`	план до выполнения
`/compact`	сжать историю
`/help`, `/exit`	все команды; выход

При смене модели история сессии сохраняется. Модель на всю сессию: `ai4science -m qwen235`.

Модели Yandex AI Studio

Алиас	Окно [49]	Назначение
qwen	256k	по умолчанию
qwen235	256k	точнее; для `/plan`
deepseek	1M	длинный контекст
gptoss	128k	короткие цепочки
junior	128k	экономия лимита

Список моделей — `ai4science models`. История сжимается при ~100k токенов для любой модели списка.

Проверка результата

Смена модели меняет порождение и не меняет проверку. Сообщение агента о выполненном действии проверяется по результату: файлу, выводу команды, записи в журнале.

Рабочая среда: агент отдельно, проект отдельно

Проект исследователя

```
my_project/
  AGENT.md     договор с агентом
  diary.md     журнал (записи добавляются)
  data/        исходные данные
  src/         код
  memory/      заметки и сводки для агента
  paper/       текст статьи
  .gitignore
```

Шаблон — в общей папке курса.

- Папка агента (`AI4Science/`) содержит программу, ключ и настройки. В проект она не копируется.
- Агент запускается *из папки проекта* и работает с её файлами.
- `AGENT.md` — договор: цель, данные, соглашения, запреты. У других агентов тот же файл называется `AGENTS.md`, `CLAUDE.md`, `.cursorrules`.
- `diary.md` — журнал: запрос, результат, точка проверки, итог.
- Проект переносится между машинами и между моделями, потому что состоит из файлов.

Git: снимки состояния проекта

```
git config --global user.name "Имя Фамилия"
git config --global user.email "you@example.org
    "

git init                    # начать историю
git add -A                  # отметить изменения
git commit -m "..."         # сохранить снимок
git log --oneline           # список снимков
git status                  # что изменилось
git diff                    # изменения построчно
git restore файл            # вернуть файл
git tag exp-01              # пометить снимок
```

- Коммит — снимок всех файлов проекта в момент времени.
- Коммит *до* задачи агенту: всё, что изменит агент, видно в `git diff` и откатывается.
- Коммит *после* проверки: принятый результат фиксируется вместе с записью в журнале.
- Метка (`tag`) указывает на снимок, по которому получен результат; по ней результат воспроизводится.
- Репозиторий можно держать локально или на любой платформе.

Техника безопасности с первого дня

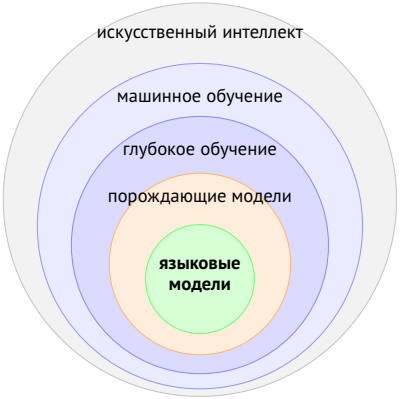
Режим	Поведение	Когда
`approve`	подтверждение каждого вызова инструмента	незнакомая задача, первые запуски
`smart_approve`	чтение — без подтверждения; запись и команды — с подтверждением	обычная работа (по умолчанию в сессии)
`auto`	все вызовы без подтверждения	короткая задача, результат которой будет проверен (`ai4science run`)
`chat`	без инструментов	обсуждение

- Агент выполняет команды в папке, из которой запущен: запуск — только из папки проекта.
- Коммит перед задачей — страховка от нежелательных изменений.
- Ключи и пароли не хранятся в папке проекта; `.env` — в `.gitignore`.
- Персональные и закрытые данные не отправляются модели без права на это.
- Внешние тексты как источник инструкций — неделя 5; недоверенные инструменты — неделя 8.

Пара 2

Языковая модель, агент, написанный и порождённый текст

Где находятся языковые модели



- **Искусственный интеллект** — область, изучающая системы, которые воспринимают среду и действуют в ней [25].
- **Машинное обучение** — построение алгоритмов по данным [24].
- **Глубокое обучение** — модели из многих слоёв преобразований.
- **Порождающие модели** — приближают распределение данных и порождают новые примеры.
- **Языковые модели** — порождающие модели текста, программ и других последовательностей символов.

Обучение по прецедентам: постановка

Стандартная постановка [24]:

- X — множество объектов, Y — множество ответов; неизвестная зависимость $y^*: X \to Y$.
- Обучающая выборка $X^\ell = (x_i, y_i)_{i=1}^\ell$, $y_i = y^*(x_i)$.
- Модель $a(x) = g(x, \theta)$ с параметрами θ; функция потерь $\mathcal{L}(a, x)$.
- Обучение: $\theta^* = \arg\min_\theta \frac{1}{\ell} \sum_{i=1}^{\ell} \mathcal{L}\big(g(\cdot, \theta), x_i\big)$.

Языковая модель в этой постановке

- Объект — префикс текста $x_{1:j} = (x_1, \ldots, x_j)$, $x_j \in \mathcal{V}$; $\mathcal{V}$ — словарь токенов.
- Ответ — следующий токен x_{j+1}; выход модели — распределение $p_\theta(\,\cdot \mid x_{1:j})$ на $\mathcal{V}$.
- Потери — $-\log p_\theta(x_{j+1} \mid x_{1:j})$; выборка — все позиции всех текстов корпуса [39].
- Разметка не нужна: ответ для каждой позиции содержится в самом тексте [27].

Токен — единица текста для модели

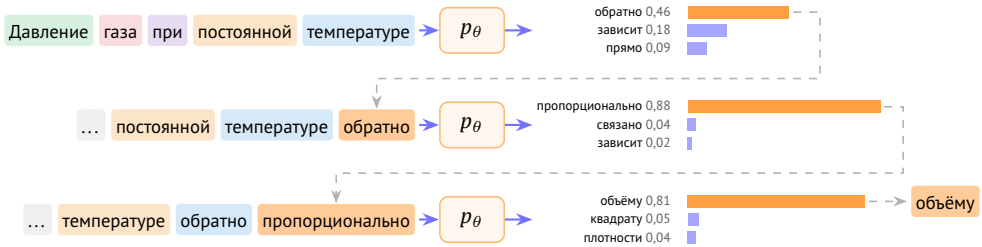
Текст разбивается на элементы словаря $\mathcal{V}$; частые сочетания символов объединены в один элемент (BPE) [26]. Ниже — разбиение токенизатором моделей Qwen3 [40], $|\mathcal{V}| = 151\,669$.

текст	токены и их номера в словаре	токенов
английский	pressure (45974) · ␣is (374) · ␣invers (64583) · ely (974) · ␣proportional (54172) · ␣to (311) · ␣volume (8123)	7
русский	дав · ление · ␣обрат · но · ␣пр · оп · ор · ци · она · ль · но · ␣объ · ё · му	14
формула	p · ␣* · ␣V · ␣== · ␣const	5
Lean	theorem · ␣foo · ␣: · ␣a · ␣+ · ␣b · ␣= · ␣b · ␣+ · ␣a · ␣:= · ␣by · ␣ring	13

- На вход модели подаётся последовательность $x_1, \ldots, x_N \in \mathcal{V}$.
  Естественный язык, формулы и код кодируются одним словарём.
- Окно контекста, лимиты и стоимость измеряются в токенах.
  Русская фраза примера заняла 14 токенов, английская того же смысла — 7.

Знак ␣ — пробел в начале токена. Разбиение вычислено токенизатором модели Qwen3-0.6B.

Авторегрессионное порождение



$$p_\theta(x_1, \ldots, x_N) \;=\; \prod_{j=0}^{N-1} p_\theta\left(x_{j+1} \mid x_{1:j}\right)$$

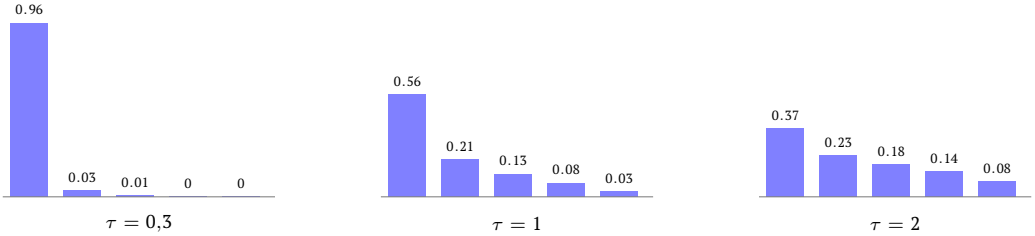
- На каждом шаге модель выдаёт распределение над $\mathcal{V}$; выбранный токен дописывается к входу, и шаг повторяется до служебного токена конца [27], [30].
- Неверное продолжение имеет ненулевую вероятность.

Для наглядности токены показаны словами; вероятности условные.

Выбор токена и температура

Модель выдаёт оценки z_v для каждого $v \in \mathcal{V}$; вероятность выбора при температуре $\tau > 0$:

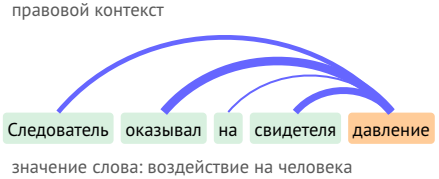
$$p_\tau(v) \;=\; \frac{\exp(z_v/\tau)}{\sum_{u\in\mathcal{V}} \exp(z_u/\tau)}.$$



- Малая τ — выбор почти всегда самого вероятного токена; большая — распределение выравнивается [31].
- Один и тот же запрос при $\tau > 0$ даёт разные продолжения: порождение недетерминировано.
- На рисунке оценки $z = (2;\ 1;\ 0{,}5;\ 0;\ -1)$ для пяти токенов.

Контекст и внимание





Упрощённо (без линейных проекций) новое представление токена j — взвешенная сумма представлений токенов префикса:

$$h'_j = \sum_{i \le j} \alpha_{ji}\, h_i, \qquad \alpha_{ji} \ge 0, \quad \sum_{i \le j} \alpha_{ji} = 1.$$

- Веса α_{ji} вычисляются по самим представлениям, поэтому один и тот же токен в разных контекстах получает разные h'_j [28], [29], [38].
- Трансформер — десятки слоёв внимания и поточечных преобразований. Модель опирается только на содержимое окна контекста; размер окна и его переполнение — неделя 3.

Толщина линии — вес α_{ji} последнего токена; значения условные.

Как модель становится собеседником

1. **Предобучение**: продолжение текстов, книг, кода; максимизация правдоподобия [27].
2. **Дообучение на инструкциях**: пары «запрос — ответ»; модель учится отвечать и заканчивать ответ [32].
3. **Обучение на предпочтениях людей** (RLHF): люди сравнивают ответы, модель учится выдавать ответы с высокой оценкой [32].

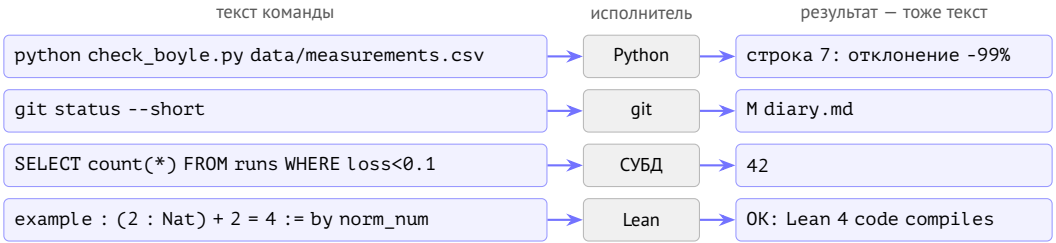
Склонность соглашаться с пользователем [33]

Пять ИИ-ассистентов, дообученных на обратной связи от людей, систематически давали ответы, согласованные с мнением пользователя, в ущерб истинности, в четырёх задачах порождения текста. Люди и модели предпочтений в заметной доле случаев выбирали убедительно написанный угодливый ответ вместо правильного.

Главное

Похвала модели не является признаком правоты. Проверка результата выполняется вне модели.

Команда — текст, исполняемый в среде



- Команда — текст, для которого есть исполнитель; результат исполнения — тоже текст. Модель, обученная на коде и документации, порождает оба вида текста одним механизмом.
- Исполнитель не оценивает смысл и цель команды: `rm -r data` исполняется так же, как `ls data`.
- В агенте текст левого столбца порождает модель, исполнителя запускает ведущее приложение, результат возвращается в окно контекста.

Вызов инструмента

1. Модель порождает вызов: фрагмент сообщения в OpenAI-совместимом интерфейсе

```
"tool_calls": [{"id": "call_1", "type": "function", "function": {
    "name": "lean_check",
    "arguments": "{\"code\": \"example : (2 : Nat) + 2 = 4 := by norm_num\"}"}}]
```

2. Ведущее приложение исполняет вызов

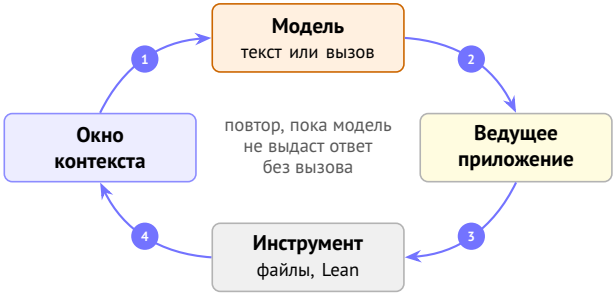
```
> lean_check ai4science-mcp
  code: example : (2 : Nat) + 2 = 4 := by norm_num
```

3. Результат дописывается в окно контекста

```
role: tool
tool_call_id: call_1
content: OK: Lean 4 code compiles
  (Mathlib imported, 156 ms).
```

- Вызов — последовательность токенов того же словаря $\mathcal{V}$; модель дообучена порождать её в строгом формате: имя инструмента и аргументы [34].
- Модель ничего не исполняет: вызов распознаёт и исполняет ведущее приложение. Результат становится частью входа на следующем шаге [35].

Цикл агента



1 вход; 2 вызов; 3 исполнение; 4 результат в окне

окно контекста после двух вызовов

AGENT.md и запрос пользователя
вызов: прочитать файл данных
результат: 12 строк таблицы
вызов: выполнить скрипт проверки
результат: строка 7: отклонение -99%
ответ без вызова

пользователь модель инструмент

- Схема «рассуждение — действие — наблюдение» [35]: наблюдение дописывается в окно контекста и влияет на следующий шаг модели.
- Ведущее приложение, режимы подтверждения, лимиты и сжатие истории — обычный код, в котором модели нет. Рост окна контекста и его стоимость — неделя 3.

Что называем агентом

Определение, принятое в курсе

Агент — автоматическая недетерминированная система с доступом к локальному контексту пользователя и к интерпретатору системных функций.

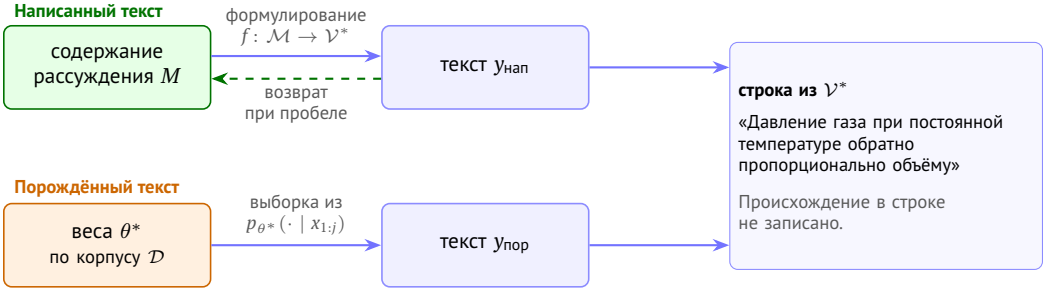
Система проявляет агентность, когда она:

- работает на **цель**: выполняет последовательность шагов ради результата;
- **выбирает действия**: инструменты, шаги, порядок;
- **использует результаты** действий для изменения дальнейших шагов;
- работает **в цикле** до достижения цели или отказа от неё.

Система	действия в среде	выбор действий	агент
чат без инструментов	нет	—	нет
скрипт с фиксированными шагами	да	нет	нет
модель, один вызов по шаблону	да	ограничен	пограничный случай
цикл с целью и инструментами	да	да	да

Агентность через цель и функциональную систему — неделя 2.

Два способа получить текст



- Написанный текст получен из содержания M; при записи автор обнаруживает пробелы и возвращается к рассуждению.
- Порождённый текст — случайная величина с распределением, заданным весами θ^* и запросом $x_{1:j}$. Содержание рассуждения пользователя входит в процесс только через запрос.
- Одна и та же строка может быть получена обоими способами.

Формальное различие

Написанный текст

$$y_{\text{нап}} = f(M), \qquad f\colon \mathcal{M} \to \mathcal{V}^*$$

$M \in \mathcal{M}$ — содержание рассуждения автора, f — формулирование.

Текст — **значение функции** от содержания.

Порождённый текст

$$y_{\text{пор}} \sim p_{\theta^*}(\,\cdot \mid x_{1:j})$$

$$\theta^* = \arg\max_{\theta} \mathbb{E}_{y \sim p_{\mathcal{D}}} \log p_\theta(y)$$

$x_{1:j}$ — запрос, $p_{\mathcal{D}}$ — распределение текстов корпуса $\mathcal{D}$.

Текст — **выборка из распределения**.

- Целевая функция обучения — правдоподобие текста относительно корпуса; истинность утверждений в неё не входит.
- В определении $y_{\text{пор}}$ нет аргумента M. Значения $y_{\text{нап}}$ и $y_{\text{пор}}$ лежат в одном множестве $\mathcal{V}^*$ и могут совпадать; различаются способы их получения.
- После дообучения на предпочтениях людей [32] в целевую функцию входят оценки людей; они не всегда совпадают с правильностью ответа [33].

M, f, y — обозначения этого раздела; $\mathcal{V}^*$ — множество конечных последовательностей токенов.

Цель обучения и неотличимость от корпуса

Максимизация правдоподобия равносильна минимизации расхождения Кульбака–Лейблера с распределением корпуса:

$$\mathbb{E}_{y\sim p_{\mathcal{D}}} \log p_\theta(y) \;=\; -\,\mathrm{KL}\big(p_{\mathcal{D}} \,\|\, p_\theta\big) + C, \qquad C \text{ не зависит от } \theta.$$

Детектор $c\colon \mathcal{V}^* \to \{0,1\}$ — любая процедура, которая по тексту решает, порождён ли он моделью. Для любого c

$$\Big|\Pr_{y\sim p_\theta}[c(y)=1] - \Pr_{y\sim p_{\mathcal{D}}}[c(y)=1]\Big| \;\le\; \mathrm{TV}\big(p_\theta, p_{\mathcal{D}}\big) \;\le\; \sqrt{\tfrac{1}{2}\,\mathrm{KL}\big(p_{\mathcal{D}} \,\|\, p_\theta\big)}.$$

Первое неравенство следует из определения TV для события $\{c(y)=1\}$, второе — неравенство Пинскера [41, лемма 2.5].

- Чем ближе модель к корпусу, тем меньше преимущество любого детектора, который видит только текст.
- Модель, близкая к корпусу, воспроизводит и его ошибки с близкой частотой. Близость к корпусу и истинность — разные свойства текста.

Логарифмы натуральные; $\mathrm{KL}(p\,\|\,q) = \sum_y p(y)\log\frac{p(y)}{q(y)}$, $\mathrm{TV}(p,q) = \sup_{A\subseteq\mathcal{V}^*} |p(A)-q(A)|$, $C = \mathbb{E}_{y\sim p_{\mathcal{D}}}\log p_{\mathcal{D}}(y)$.

Истинность как отношение структуры и формулы

А. Тарский определил истинность формулы через её выполнимость при интерпретации символов языка [42]. В теории моделей истинность — отношение между **структурой** $\mathfrak{M}$ и формулой φ [43]:

$$\mathfrak{M} \models \varphi \quad \Longleftrightarrow \quad \varphi \text{ истинна в структуре } \mathfrak{M}.$$

Структура — непустое множество (носитель) и значения символов языка на нём. В логике $\mathfrak{M}$ называют моделью; в курсе — структурой, чтобы не путать с языковой моделью.

Формула $\varphi := \forall x \exists y\, (x \cdot y = 1)$

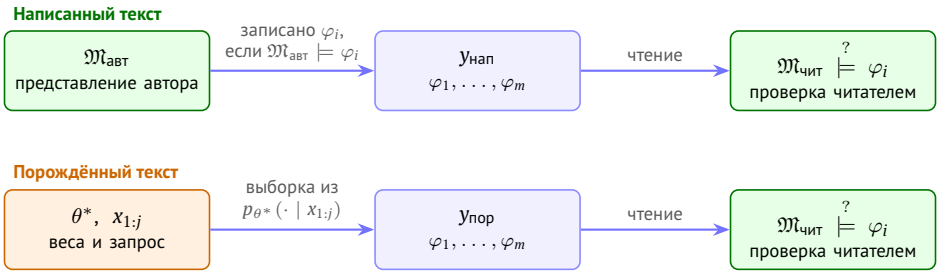
Структура	**Носитель**	**Истинность** φ
$\mathfrak{M}_1 = (\mathbb{N},\ \cdot,\ 1)$	натуральные числа	$\mathfrak{M}_1 \not\models \varphi$: для $x = 2$ нет y
$\mathfrak{M}_2 = (\mathbb{Q}_{>0},\ \cdot,\ 1)$	положительные рациональные числа	$\mathfrak{M}_2 \models \varphi$: $y = 1/x$

Главное

Одна и та же строка φ истинна в одной структуре и ложна в другой. Значение истинности имеет пара $(\mathfrak{M}, \varphi)$.

Интерпретация текста: автор и читатель

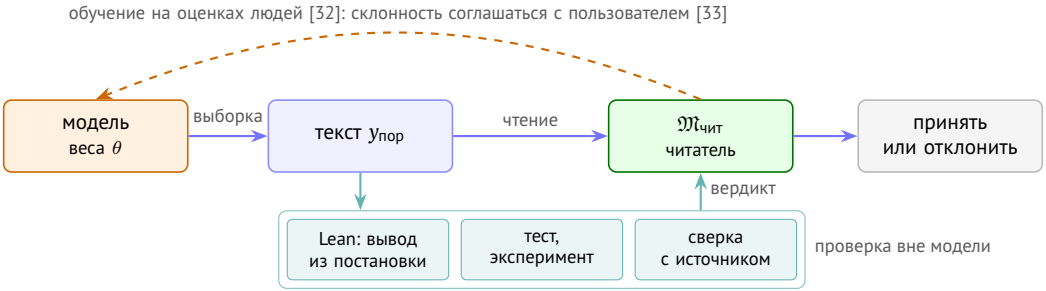
Утверждения текста — $\varphi_1, \ldots, \varphi_m$; представления предметной области у автора и у читателя — структуры $\mathfrak{M}_{\text{авт}}$ и $\mathfrak{M}_{\text{чит}}$.



- Написанный текст интерпретирован дважды: автор записал φ_i, считая $\mathfrak{M}_{\text{авт}} \models \varphi_i$, читатель проверяет его в $\mathfrak{M}_{\text{чит}}$. Расхождение указывает на ошибку одного из двух.
- В процедуре выборки нет шага, на котором для какой-либо структуры вычисляется $\mathfrak{M} \models \varphi_i$. Без внешних инструментов первая оценка истинности порождённого текста происходит при чтении.

Схема-аналогия: утверждения естественного языка формулами не являются, представления людей — не структуры в строгом смысле. Схема показывает, в какой момент появляется оценка истинности.

Читатель как проверяющая структура



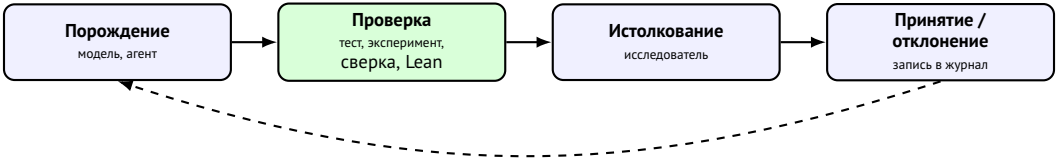
- **Иллюзия авторства**: согласие текста с $\mathfrak{M}_{\text{чит}}$ воспринимается как понимание автора; одобренный порождённый текст легко принять за собственную мысль.
- Внешняя проверка не зависит от $\mathfrak{M}_{\text{чит}}$: Lean проверяет вывод относительно записанной постановки, тест и эксперимент — поведение и данные, сверка — факт в источнике.
- Постановка, мотивация, новизна и истолкование результата формально не проверяются; для них остаётся критическое чтение.

Четыре вида текста в работе исследователя

Вид	Что это	Как с ним обращаться
Написанный	след рассуждения автора	авторская ответственность
Порождённый	выборка из модели; уверенный тон задаётся грамматикой	проверить, прежде чем использовать
Принятый	порождённый текст, прошедший проверку и понятый исследователем	становится написанным: автор отвечает за него
Исполняемый	текст, который исполнитель приводит в действие: код, команда, Lean	результат исполнения — источник проверки

- 2+2 и `"2+2"` — разные объекты: первое вычисляется, второе хранится.
- Агент постоянно переводит текст из одного вида в другой: порождает команды, исполняет их, пишет отчёт.
- В отчётах и статье курса виды отмечаются: `[порождено]`, `[написано]`, `[источник]`.

Петля недоверия



- Любой результат модели проходит одну и ту же последовательность: порождение, проверка, истолкование, принятие или отклонение.
- Проверка выполняется вне модели. Истолкование и решение остаются за исследователем.
- Отклонённый результат возвращается в цикл с новой информацией: ошибкой, контрпримером, замечанием.
- На каждом модуле курса добавляется свой уровень проверки: данные и код (H6), Lean (H7), критик (H9), рецензия (H11).

Способы проверки

Способ	Что даёт	Неделя
Формальное доказательство	двоичный вердикт для формализуемого ядра	7
Численный эксперимент с планом	быстрый контрпример; оценка на наборе условий	6
Код и тесты	проверка поведения на заданных входах и свойствах	6
Проверка данных	схема, единицы, диапазоны, повторяемость	6
Сверка с базой и источником	наличие факта в источнике	5, 8
Человек	постановка, мотивация, новизна, истолкование	все

- Способы расположены по силе гарантии; чем выше способ, тем уже область, где он применим.
- В демонстрации с законом Бойля–Мариотта проверка данных подтвердила числа и опровергла объяснение агента.

Три режима использования агента

Зона ближайшего развития — расхождение между уровнем задач, которые человек решает самостоятельно, и уровнем задач, которые он решает в сотрудничестве [36, гл. 6].

Режим	Роль агента	Риск
«Знаю»	оформитель: переписать, отформатировать, повторить известное	низкий
«Знаю, что не знаю»	собеседник: вопросы, варианты, объяснения; исследователь может оценить ответ	средний
«Не знаю, что не знаю»	у исследователя нет образа результата, с которым сравнить ответ	высокий

Главное

Без образа ожидаемого результата сравнивать ответ агента не с чем. В третьем режиме сначала формулируется, как будет проверен результат, затем даётся задача.

Триада: что остаётся у исследователя

- **Инференс** меняется: модели обновляются, поставщики сменяются, часть задач переходит на локальные модели (неделя 12).
- **Контекст** — `AGENT.md`, данные, онтология, журнал, сводки — принадлежит исследователю и переносится между моделями.
- **Проверка** — тесты, скрипты, формальные постановки — тоже файлы проекта.

Аналогия

В 1970-е годы вычисления выполнялись на мейнфреймах коллективного пользования; затем распространились персональные компьютеры, настроенные под задачи владельца. Курс рассматривает похожий переход: от универсального облачного чата к настроенной локальной среде исследователя.

Главное

Среда состоит из файлов проекта. Она сохраняет ценность при смене модели и без облачного сервиса.

Самостоятельная практика: артефакт недели 1

Что сделать: среда на своём ноутбуке в git-репозитории и цикл «порождение → проверка» с явной точкой проверки.

Шаги (сценарий — в общей папке курса, файл `practice.md`):

1. Проверить git и Python 3.10+; установить агента; `ai4science doctor`.
2. Создать проект из шаблона, заполнить цель и данные в `AGENT.md`, первый коммит.
3. Дать агенту небольшую задачу по своему материалу в режиме `approve`.
4. Проверить результат без агента: пересчёт, запуск, сверка с источником.
5. Записать в `diary.md`: запрос, результат, точку проверки, итог; коммит.

Минимум для «зачтено»: `doctor` без ошибок; проект в git с `AGENT.md` и `diary.md`; запись в журнале с заполненной точкой проверки.
Время: около 3 ч. **Идёт в:** ДЗ 1.

Домашняя работа 1: объявление

ДЗ 1. Среда и контекст — недели 1–5, сдача к неделе 6, 20 баллов.

- Личная среда, в которой агент работает с вашим материалом: договор `AGENT.md`, небольшая онтология и граф понятий темы, поиск по личной библиотеке, журнал работы.
- Складывается из артефактов недель 1–5.
- Приёмка: отчёт с приложенным `AGENT.md` и примерами запросов к локальному контексту.
- Постановка и критерии оценивания выдаются на неделе 2.

До недели 2: установить агента, создать проект, выполнить первый цикл. Трудности установки — в чат курса; разбор типовых ошибок откроет следующую лекцию.

Итог недели 1

1. Порождение результатов ИИ-системами стало дешёвым; результаты, прошедшие исполнимую проверку, защищены сильнее остальных [1], [4], [13], [17].
2. Всё, с чем работает исследователь, можно представить текстовыми файлами: их читает агент, хранит git, проверяют программы.
3. Языковая модель порождает следующий токен по распределению $p_\theta(x_{j+1} \mid x_{1:j})$; при $\tau > 0$ порождение недетерминировано.
4. Агент — цикл: модель порождает вызов, ведущее приложение выполняет его в среде, результат возвращается в контекст.
5. Написанный текст — след рассуждения автора; порождённый — выборка из распределения, близкого к корпусу. Истинность — отношение текста и структуры [42]: для порождённого текста её впервые оценивает читатель или внешняя проверка.
6. Петля недоверия: порождение, проверка, истолкование, принятие или отклонение. Похвала модели признаком правоты не является [33].

Мостик к неделе 2

- Сегодня агент описан как цикл «модель — вызов — среда — результат».
- Неделя 2: что делает этот цикл целенаправленным. Агент как функциональная система: цель, эталон результата, сличение, действие.
- Текст и команда как две знаковые системы с общим алфавитом; три истолкователя: человек, исполнитель, модель.
- Агент в коде: простейший цикл на Python, который можно прочитать целиком.

Главное

К неделе 2 у каждого: работающий агент, проект в git, первая запись в журнале с точкой проверки.

Литература недели 1 I

[1] T. Hubert *et al.*, "Olympiad-level formal mathematical reasoning with reinforcement learning," *Nature*, vol. 651, no. 8106, pp. 607–613, 2026, doi: 10.1038/s41586-025-09833-y.

[2] T. Luong and E. Lockhart, "Advanced version of Gemini with Deep Think officially achieves gold-medal standard at the International Mathematical Olympiad," Google DeepMind Blog, Jul. 21, 2025.

[3] A. Novikov *et al.*, "AlphaEvolve: A coding agent for scientific and algorithmic discovery," arXiv:2506.13131, 2025.

[4] B. Georgiev, J. Gómez-Serrano, T. Tao, and A. Z. Wagner, "Mathematical exploration and discovery at scale," arXiv:2511.02864, 2025.

[5] A. Zhou and R. Arel, "Tempest: Autonomous multi-turn jailbreaking of large language models with tree search," arXiv:2503.10619v5, 2025.

[6] Intology, "Zochi publishes A* paper," Intology Blog, May 27, 2025. `https://intology.ai/blog/zochi-acl`

[7] ACL Rolling Review, "Call for papers." `https://aclrollingreview.org/cfp` (accessed Sep. 14, 2026).

[8] T. Achim *et al.*, "Aristotle: IMO-level automated theorem proving," arXiv:2510.01346, 2025.

[9] B. Alexeev, "Formalization of Erdős problems," *Xena Project* (blog), Dec. 5, 2025.

[10] N. Shmatko, A. Adam, P. Esau, A. Cui, and E. Tian, "GPTZero finds 100 new hallucinations in NeurIPS 2025 accepted papers," GPTZero, Jan. 21, 2026.

[11] NeurIPS 2025 Program Committee Chairs, "Reflections on the 2025 review process from the program committee chairs," NeurIPS Blog, Sep. 30, 2025.

[12] W. Liang *et al.*, "Monitoring AI-modified content at scale: A case study on the impact of ChatGPT on AI conference peer reviews," in *Proc. ICML*, PMLR vol. 235, 2024, pp. 29575–29620.

[13] D. E. Knuth, "Claude's cycles," Stanford Univ., Feb. 28, 2026 (rev. Apr. 14, 2026).

[14] K. Morrison, "KnuthClaudeLean," GitHub repository, 2026. `https://github.com/kim-em/KnuthClaudeLean`

[15] OpenAI, "On the Navier–Stokes Millennium Prize Problem," Sep. 8, 2026 (updated Sep. 10, 2026). `https://openai.com/index/navier-stokes-solution/`

[16] OpenAI, "Finite time blowup for Navier–Stokes," preprint, 166 pp., Sep. 2026.

[17] OpenAI, "NavierStokesAndEuler: Lean certificates accompanying Navier–Stokes and Euler results," GitHub repository, Sep. 2026.

[18] C. L. Fefferman, "Existence and smoothness of the Navier–Stokes equation," in *The Millennium Prize Problems*, Clay Math. Inst., 2006, pp. 57–67.

[19] Clay Mathematics Institute, "Navier-Stokes announcement," Sep. 11, 2026.

[20] L. Alpöge and T. Buckmaster, "Blowup for the Euler equations with smooth forcing," preprint, Sep. 2026.

[21] T. Tao, posts on Mathstodon, Sep. 3–11, 2026.

[22] A. Avila *et al.*, "A severe misalignment of AI in mathematics," declaration, Sep. 2026. `https://mathandai.org/`

Литература недели 1 II

[23] Clay Mathematics Institute, "Rules for the Millennium Prize Problems," rev. Sep. 26, 2018.

[24] К. В. Воронцов, «Машинное обучение (курс лекций)», MachineLearning.ru. http://www.machinelearning.ru/

[25] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[26] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proc. ACL (Vol. 1: Long Papers)*, 2016, pp. 1715–1725, doi: 10.18653/v1/P16-1162.

[27] A. Radford *et al.*, "Language models are unsupervised multitask learners," OpenAI, Tech. Rep., 2019.

[28] A. Vaswani *et al.*, "Attention is all you need," in *Adv. NeurIPS*, vol. 30, 2017, pp. 5998–6008.

[29] J. Alammar, "The illustrated Transformer," 2018. https://jalammar.github.io/illustrated-transformer/

[30] J. Alammar, "The illustrated GPT-2," 2019. https://jalammar.github.io/illustrated-gpt2/

[31] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," in *Proc. ICLR*, 2020.

[32] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," in *Adv. NeurIPS*, vol. 35, 2022, pp. 27730–27744.

[33] M. Sharma *et al.*, "Towards understanding sycophancy in language models," in *Proc. ICLR*, 2024, pp. 110–144.

[34] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," in *Adv. NeurIPS*, vol. 36, 2023, pp. 68539–68551.

[35] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[36] Л. С. Выготский, *Мышление и речь. Психологические исследования*. М.; Л.: Соцэкгиз, 1934.

[37] goose: an open source, extensible AI agent, GitHub repository, v1.50.0, 2026. https://github.com/aaif-goose/goose

[38] К. В. Воронцов, «Модели внимания и трансформеры», слайды курса «Машинное обучение», МФТИ, 27 февр. 2026.

[39] К. В. Воронцов, «Лекция 3. Нейросетевые языковые модели», курс «Вероятностные языковые модели», ВМК МГУ, 16 марта 2026.

[40] A. Yang *et al.*, "Qwen3 technical report," arXiv:2505.09388, 2025.

[41] A. B. Tsybakov, *Introduction to Nonparametric Estimation*. New York, NY, USA: Springer, 2009, doi: 10.1007/b13794.

[42] A. Tarski, "The concept of truth in formalized languages," in *Logic, Semantics, Metamathematics*, J. H. Woodger, Trans. Oxford, U.K.: Clarendon Press, 1956, pp. 152–278.

[43] A. Tarski and R. L. Vaught, "Arithmetical extensions of relational systems," *Compositio Mathematica*, vol. 13, no. 2, pp. 81–102, 1957.

[44] A. J. Ayer, *Language, Truth and Logic*. London, U.K.: Victor Gollancz, 1936.

[45] T. Uebel, "Vienna Circle," in *The Stanford Encyclopedia of Philosophy*, E. N. Zalta and U. Nodelman, Eds., rev. May 7, 2024. https://plato.stanford.edu/entries/vienna-circle/

[46] K. Popper, *Logik der Forschung*. Wien, Austria: Julius Springer, 1935; англ. пер.: *The Logic of Scientific Discovery*. London, U.K.: Hutchinson, 1959.

[47] W. V. O. Quine, "Two dogmas of empiricism," *The Philosophical Review*, vol. 60, no. 1, pp. 20–43, 1951.

[48] A. Comte, *Cours de philosophie positive*, 6 vols. Paris, France: Bachelier, 1830–1842.

[49] Yandex Cloud, "Available generative models," Yandex AI Studio documentation. https://aistudio.yandex.ru/docs/en/ai-studio/concepts/generation/models.html (accessed Sep. 14, 2026).